UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería de Sistemas y Electrónica

Curso: Servicios Cloud

INFORME DE LABORATORIO N.° 10

Monitoreo en la nube y gestión de métricas en bases de datos de series temporales

Análisis de un incidente de degradación de una dependencia con Prometheus, PromQL y Grafana

Docente	Dr. Eleodoro Orbegoso Carrera
Estudiante	Icochea Rodriguez, Thiago Paolo
Sección	45104
Equipo	Trabajo individual
Número de laboratorio	10
Fecha de realización	16/10/2026
Repositorio	https://github.com/ThiagoIcochea/LaboratorioSemana10

Lima – Perú, 2026

Contenido

1. Resumen ejecutivo..... 4
2. Objetivos..... 4
2.1. Objetivo general..... 4
2.2. Objetivos específicos..... 4
3. Fundamento conceptual..... 4
4. Entorno y arquitectura..... 6
4.1. Entorno de ejecución..... 6
4.2. Arquitectura implementada y flujo de métricas..... 7
4.3. Configuraciones esenciales..... 7
5. Procedimiento..... 8
6. Resultados..... 9
6.1. Inventario de métricas..... 9
6.2. Línea base..... 10
6.3. Incidente..... 11
6.4. Recuperación..... 15
6.5. Consultas PromQL..... 16
6.6. Alertas..... 17
7. Análisis del fallo..... 18
7.1. Línea temporal..... 18
7.2. Hoja de análisis del incidente..... 19
7.3. Correlación de señales y contraste de hipótesis..... 20
7.4. Diagnóstico argumentado..... 21
7.5. Limitaciones del análisis..... 21
8. Propuesta de mejora..... 21
8.1. Acción correctiva inmediata..... 21
8.2. Medidas preventivas..... 21
8.3. Reglas de alerta y umbrales justificados..... 22
8.4. Métricas de seguimiento y verificación de la recuperación..... 22
9. Conclusiones..... 22
10. Referencias bibliográficas..... 23
11. Cierre y metacognición..... 23
12. Autoevaluación frente a la rúbrica..... 24

13. Anexos..... 24
Anexo A. Consultas PromQL completas..... 24
Anexo B. Reglas de registro y de alerta (prometheus/alerts.yml)..... 25
Anexo C. Fragmento de instrumentación y modo de fallo (application/app.py)..... 26
Anexo D. Scripts y datos exportados..... 27
Anexo E. Evidencias complementarias..... 27

1. Resumen ejecutivo

Este laboratorio tuvo como objetivo comprender y aplicar el monitoreo en la nube mediante métricas almacenadas en una base de datos de series temporales. Se implementó una aplicación web instrumentada, node_exporter, Prometheus 3.5.0 como recolector y TSDB, reglas de alerta y un dashboard de Grafana provisionado, empaquetados con Docker Compose para GitHub Codespaces. Con tráfico constante de 8 peticiones por segundo se registró una línea base, luego se degradó de forma controlada la dependencia interna inventario-db durante cinco minutos y finalmente se verificó la recuperación. Durante el incidente, la latencia p95 pasó de 54,7 ms a 2,20 s y los errores 5xx de 0,04 % a 32,4 %, mientras la CPU del host se mantuvo por debajo del 4 % y la memoria disponible sobre el 92 %. La correlación de estas señales descartó la presión de recursos y señaló a la dependencia como causa. Las alertas dispararon unos 75 segundos después del inicio. Se recomienda aplicar timeouts y un cortacircuitos hacia la dependencia, además de alertar sobre su latencia.

2. Objetivos

2.1. Objetivo general

Explicar los sistemas de monitoreo en la nube y analizar la gestión de métricas en bases de datos de series temporales mediante un caso de fallo de infraestructura reproducido de forma controlada (logro de la sesión).

2.2. Objetivos específicos

- Implementar una arquitectura de monitoreo reproducible (fuente de métricas → Prometheus/TSDB → PromQL → Grafana → alertas) y explicar la función de cada componente.
- Construir un inventario de métricas reales con su significado, unidad, etiquetas y comportamiento esperado.
- Registrar la línea base del servicio y compararla con el intervalo de incidente y de recuperación mediante consultas PromQL.
- Contrastar hipótesis (CPU, memoria, E/S, dependencia) correlacionando al menos tres señales temporales.
- Proponer acciones correctivas, preventivas y reglas de alerta con umbrales justificados por los datos.

Tabla 1. *Correspondencia entre los resultados de aprendizaje y las evidencias*

Resultado específico de la guía	Evidencia en este informe
Explica la arquitectura de un sistema de monitoreo en la nube	Sección 4, Figura 1 y Tabla 3
Distingue métricas de infraestructura y su dimensión temporal	Sección 6.1, Tabla 5 (inventario)
Analiza un fallo con datos temporales	Secciones 6.2–6.6 y 7 (consultas, figuras, hoja de análisis)
Propone acciones de respuesta y prevención	Sección 8 y reglas de alerta del Anexo B

3. Fundamento conceptual

Tabla 2. *Conceptos fundamentales del laboratorio*

Concepto	Explicación
Monitoreo	Proceso de obtener y seguir señales cuantitativas del

Concepto	Explicación
	comportamiento de aplicaciones e infraestructura para detectar desviaciones y apoyar decisiones operativas. Beyer et al. (2016) proponen vigilar cuatro «señales doradas»: latencia, tráfico, errores y saturación; este laboratorio mide las cuatro.
Observabilidad	Capacidad de explicar el estado interno de un sistema a partir de los datos que emite (métricas, registros y trazas). A diferencia del monitoreo de umbrales conocidos, permite investigar fallos no previstos formulando nuevas preguntas sobre la telemetría (Majors et al., 2022).
Métrica	Medición numérica observada en el tiempo. Prometheus define cuatro tipos: contador (solo crece, p. ej. peticiones totales), gauge (sube y baja, p. ej. peticiones en curso), histograma (distribución en cubetas, p. ej. latencias) y resumen (Prometheus Authors, 2026b).
Serie temporal	Secuencia de muestras (marca de tiempo, valor). En Prometheus cada serie se identifica de forma única por el nombre de la métrica y su conjunto de etiquetas clave/valor; por ejemplo, `app_http_requests_total{endpoint="/api/checkout",status="503"}` es una serie distinta de la de estado 200 (Prometheus Authors, 2026b).
TSDB	Base de datos optimizada para datos indexados por tiempo y consultas por rangos. Prometheus incluye una TSDB local que agrupa las muestras en bloques de dos horas, usa un registro de escritura anticipada (WAL) para tolerar caídas y aplica una política de retención (Prometheus Authors, 2026d). En el laboratorio la retención es de 2 días.
Prometheus	Sistema de monitoreo de código abierto que recolecta métricas mediante el modelo pull: consulta periódicamente el endpoint HTTP /metrics de cada objetivo, almacena las muestras en su TSDB y evalúa reglas de registro y de alerta (Prometheus Authors, 2026a).
PromQL	Lenguaje funcional de consulta de Prometheus. Opera sobre vectores instantáneos y de rango; funciones como `rate()` convierten contadores en tasas por segundo, `histogram_quantile()` estima percentiles a partir de cubetas y los agregadores (`sum by`, `avg by`) resumen conservando dimensiones (Prometheus Authors, 2026c).
Grafana	Plataforma de visualización que se conecta a Prometheus como fuente de datos, ejecuta consultas PromQL y presenta los resultados en paneles (Grafana Labs, 2026a).
Dashboard	Conjunto organizado de paneles que permite observar tendencias, picos y relaciones entre señales en una misma ventana de tiempo. En este laboratorio se provisiona como código (JSON), lo que lo hace reproducible (Grafana Labs, 2026b).
Alertamiento	Reglas que evalúan una condición PromQL de forma periódica; si se cumple durante el tiempo indicado en `for`, la alerta pasa de pending a firing (Prometheus Authors, 2026e). Una alerta indica que algo requiere atención; no sustituye el análisis causal.

Fuente: elaboración propia a partir de las referencias citadas.

4. Entorno y arquitectura

4.1. Entorno de ejecución

El proyecto se diseñó para **GitHub Codespaces**, el entorno de desarrollo en la nube de GitHub que ejecuta un contenedor de desarrollo definido en `.devcontainer/devcontainer.json` (GitHub, 2026). La configuración usa la imagen oficial de Python con la característica *Docker-in-Docker*, solicita 2 núcleos y 4 GB (dentro de la cuota gratuita) y reenvía los puertos 8000, 9090 y 3000. Con **Docker Compose** (Docker Inc., 2026) se levantan cuatro servicios: app, node-exporter, prometheus y grafana.

Entorno donde se obtuvieron los resultados. La ejecución real se realizó en una máquina virtual Linux en la nube (2 vCPU, 7,8 GiB de RAM) asociada a la sesión de trabajo. Ese entorno no podía descargar imágenes de Docker Hub ni el instalador de Grafana (la red las bloqueaba), por lo que se ejecutaron como procesos los **binarios oficiales** de Prometheus 3.5.0 y node_exporter 1.9.1 publicados en GitHub, con la misma configuración del repositorio (`scripts/run_local_sin_docker.sh`). Grafana no pudo ejecutarse allí: su datasource y su dashboard quedaron provisionados como código y se ejecutan con `docker compose up` en Codespaces. Por eso las figuras de este informe provienen de la interfaz real de Prometheus y de gráficos construidos con los datos exportados de su API; no se presentan capturas de Grafana simuladas. No se usaron servicios de pago.

Tabla 3. *Componentes, nombres lógicos y versiones (sin credenciales)*

Componente	Nombre lógico / dirección	Versión	Función en el flujo
App «catalogo»	app:8000 (localhost:8000)	Python 3, Flask 3.1.2, prometheus-client 0.23.1, Gunicorn 23.0.0	Fuente de métricas de aplicación (/metrics)
Dependencia inventario-db	interna a la app	simulada	Origen del fallo controlado
node_exporter	node-exporter:9100	1.9.1	Exportador de métricas del host
Prometheus	prometheus:9090	3.5.0	Recolección (pull, 5 s), TSDB, PromQL, reglas
Grafana	grafana:3000	12.1.1 (provisionado)	Visualización (dashboard de 12 paneles)
Generador de tráfico	scripts/loadgen.py	—	8 req/s: 70 % productos, 20 % checkout, 10 % raíz

Fuente: elaboración propia. Los nombres lógicos corresponden a Docker Compose; entre paréntesis, la dirección usada en la ejecución.

4.2. Arquitectura implementada y flujo de métricas

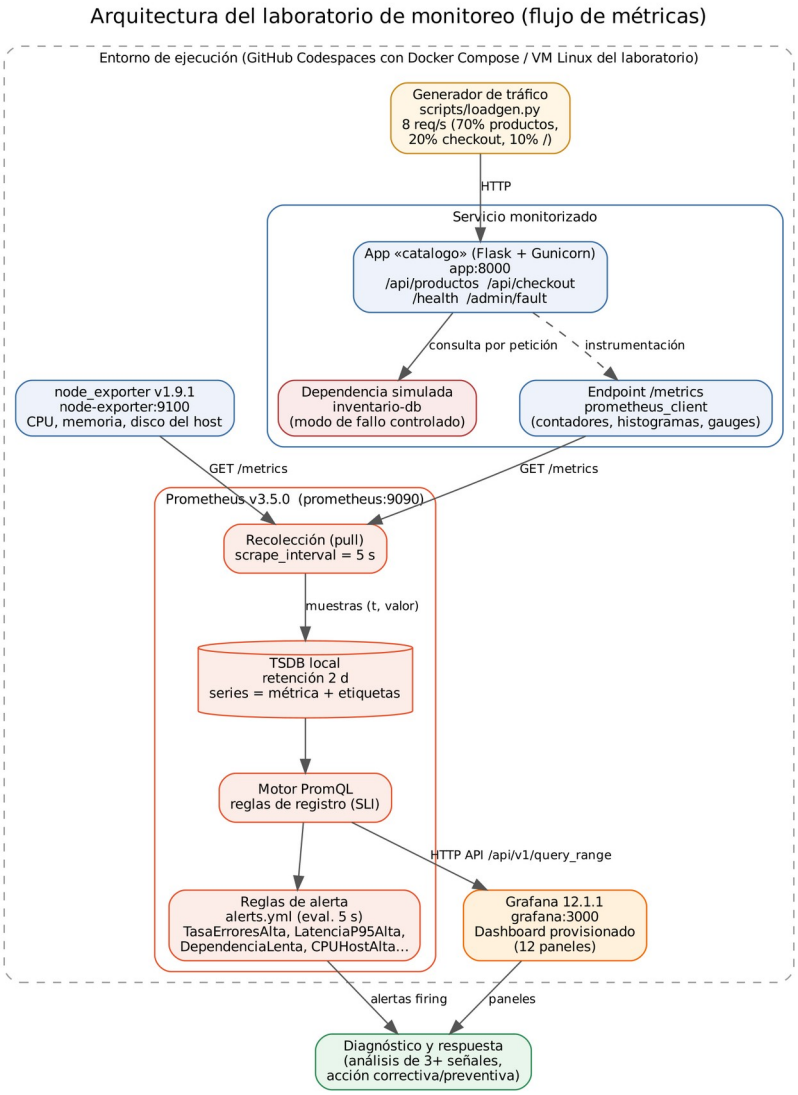


Figura 1. *Arquitectura del laboratorio y flujo de métricas*

Fuente: elaboración propia (Graphviz, evidencias/arquitectura/arquitectura.dot).

Interpretación: el generador produce tráfico HTTP hacia la app; cada petición consulta a inventario-db. La app y node_exporter exponen /metrics, que Prometheus recolecta cada 5 s (modelo pull) y guarda en su TSDB. El motor PromQL alimenta las reglas de registro y de alerta, y Grafana consulta la API de Prometheus. Las alertas y paneles sirven de entrada al diagnóstico.

La aplicación expone métricas siguiendo las recomendaciones de instrumentación: un contador de peticiones con etiquetas `method`, `endpoint` y `status`; un histograma de latencia por endpoint; un histograma y un contador de errores de la dependencia; y un gauge de concurrencia. El modo de fallo se activa con `POST /admin/fault?enable=1` y solo afecta a inventario-db: añade una latencia de 0,6 a 1,4 s (con `time.sleep`, que **no consume CPU**) y un 30 % de errores por timeout. La ruta raíz `/` no consulta la dependencia, lo que sirve como grupo de control.

4.3. Configuraciones esenciales

```
# prometheus/prometheus.yml (extracto)
global:
  scrape_interval: 5s        # resolución para ver incidentes de pocos minutos
```

```
  evaluation_interval: 5s
rule_files: [/etc/prometheus/alerts.yml]
scrape_configs:
  - job_name: catalogo-app
    static_configs: [{targets: ["app:8000"], labels: {service: catalogo}}]
  - job_name: node
    static_configs: [{targets: ["node-exporter:9100"]}]
  - job_name: prometheus
    static_configs: [{targets: ["prometheus:9090"]}]
```

Se eligió un intervalo de 5 s porque el incidente dura cinco minutos: genera 60 muestras por serie durante el fallo sin sobrecargar el entorno (el proceso de la app consumió ≈ 0,012 núcleos). Los almacenes de Prometheus y Grafana usan volúmenes de Docker y la contraseña de Grafana se toma de una variable de entorno, no del repositorio.

5. Procedimiento

Tabla 4. *Secuencia reproducible ejecutada (hora de Lima, 16/10/2026)*

N.°	Paso	Acción y verificación
1	Verificación de herramientas	Se comprobó la disponibilidad de Docker 29.8.2, Docker Compose 5.5.1, Python 3 y Chromium sin interfaz. La descarga de imágenes desde Docker Hub y de Grafana fue rechazada por la red (HTTP 403), por lo que se usó el modo sin Docker descrito en 4.1.
2	Construcción del proyecto	Se crearon la app instrumentada, `prometheus.yml`, `alerts.yml`, el dashboard y el datasource de Grafana, `docker-compose.yml`, el devcontainer y los scripts de cada fase.
3	Validación de configuración	`promtool check config` y `promtool check rules` devolvieron SUCCESS (10 reglas).
4	Arranque y validación	Se ejecutó `scripts/run_local_sin_docker.sh` y `scripts/validar_entorno.sh`: los tres objetivos quedaron en estado UP, la última muestra tenía 4,9 s de antigüedad y las reglas en estado inactive/ok.
5	Calentamiento	60 s de tráfico (18:30:47) para llenar las ventanas de `rate()`.
6	Línea base	`ejecutar_baseline.sh 360`: tráfico normal de 18:31:47 a 18:37:48.
7	Incidente	`ejecutar_incidente.sh 300`: activa el fallo a las 18:37:48, mantiene el mismo tráfico y lo desactiva a las 18:42:50.
8	Recuperación	`ejecutar_recuperacion.sh 360`: tráfico normal de 18:42:50 a 18:48:50.
9	Exportación y evidencias	`exportar_metricas.sh` (series cada 5 s, 57 consultas PromQL por fase, transiciones de alertas), `graficar.py` (gráficos) y `capturas_prometheus.py` (capturas de la interfaz real).

Comandos para reproducir en GitHub Codespaces:

```
docker compose up -d --build
bash scripts/validar_entorno.sh
bash scripts/ejecutar_experimento.sh      # base 360 s -> incidente 300 s ->
recuperación 360 s -> exportación
python3 scripts/graficar.py && python3 scripts/capturas_prometheus.py
python3 scripts/generar_informe.py
docker compose down -v                    # y detener el Codespace
```

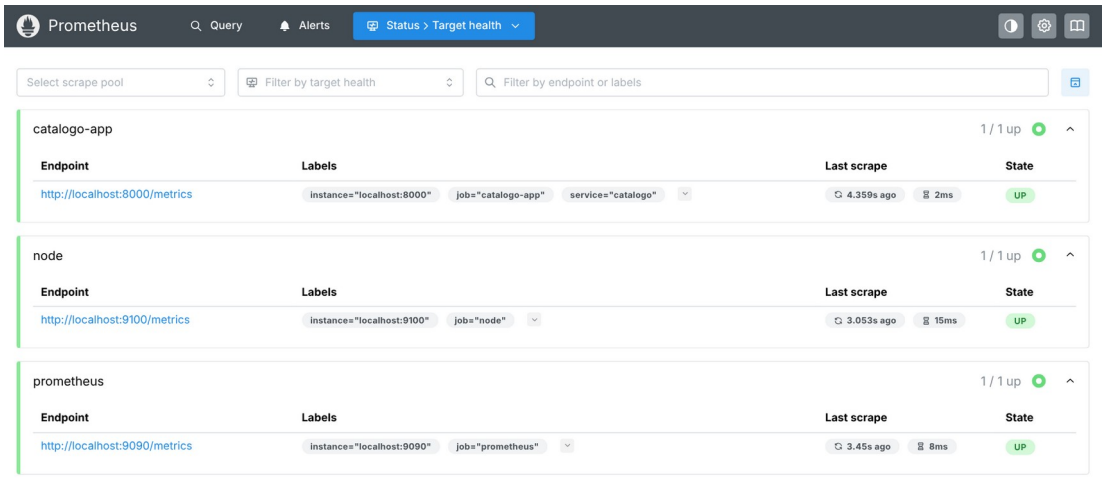


Figura 2. *Objetivos de recolección de Prometheus en estado UP*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: los tres objetivos (catalogo-app, node y prometheus) se recolectan correctamente y sin errores, lo que confirma que existen datos recientes antes de iniciar las mediciones.

6. Resultados

6.1. Inventario de métricas

Tabla 5. *Inventario de métricas reales expuestas*

Métrica	Qué representa	Unidad / tipo	Etiquetas relevantes	Comportamiento esperado
`up`	Si Prometheus alcanzó el objetivo en el último scrape	0/1 (gauge sintético)	job, instance	Constante en 1
`app_http_requests_total`	Peticiones HTTP atendidas	peticiones (contador)	method, endpoint, status	Crece ≈ 8/s; 5xx casi nulos
`app_http_request_duration_seconds`	Distribución de la latencia de respuesta	segundos (histograma)	endpoint, le	p95 < 100 ms
`app_dependency_duration_seconds`	Latencia de llamadas a inventario-db	segundos (histograma)	dependency, le	p95 ≈ 35–50 ms
`app_dependency_errors_total`	Errores de la dependencia	errores (contador)	dependency, reason	≈ 0,2 % transitorios
`app_http_inflight_requests`	Peticiones en curso	peticiones (gauge)	—	0–2
`node_cpu_seconds_total`	Tiempo de CPU por modo	segundos (contador)	cpu, mode	idle ≈ 95 %

Métrica	Qué representa	Unidad / tipo	Etiquetas relevantes	Comportamiento esperado
`node_memory_MemAvailable_bytes`	Memoria disponible del host	bytes (gauge)	instance	> 85 % del total
`node_disk_io_time_seconds_total`	Tiempo de disco ocupado	segundos (contador)	device	Cercano a 0
`process_cpu_seconds_total`	CPU consumida por el proceso de la app	segundos (contador)	job	≈ 0,01 núcleos
`process_resident_memory_bytes`	Memoria residente de la app	bytes (gauge)	job	≈ 30–35 MB estable
`app_fault_injection_active`	Marcador del experimento (no se usa como evidencia diagnóstica)	0/1 (gauge)	—	0 salvo en el incidente

Fuente: elaboración propia a partir del endpoint /metrics de la app y de node_exporter.

Para el incidente se seleccionaron cinco métricas relacionadas: latencia de la dependencia, latencia HTTP, tasa de errores, CPU del host y memoria disponible. Las dos últimas sirven para contrastar la hipótesis de presión de recursos sugerida por la guía.

6.2. Línea base

Intervalo: **2026-10-16 18:31:47 a 18:37:48** (hora de Lima; 361 s). Con tráfico constante, el servicio atendió ≈ 2 886 peticiones (7,99 req/s), con latencia media de 24,1 ms, p95 de 54,7 ms y 0,035 % de respuestas 5xx (un error transitorio de la dependencia). La CPU del host promedió 3,8 %, la memoria disponible no bajó del 90,1 % y la concurrencia máxima fue 1. Este es el patrón de referencia.

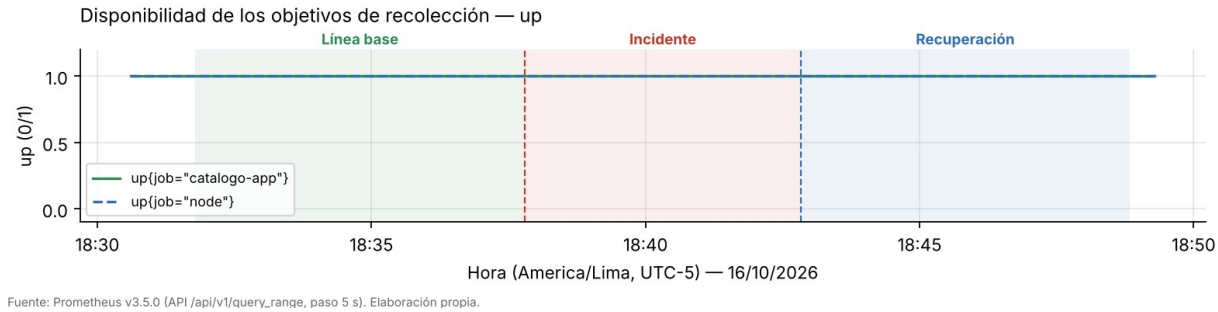


Figura 3. *Disponibilidad de los objetivos durante todo el experimento (up)*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: up permaneció en 1 para la app y para node_exporter en las tres fases. El objetivo nunca dejó de responder a Prometheus, por lo que el incidente no fue una caída del servicio, sino una degradación.

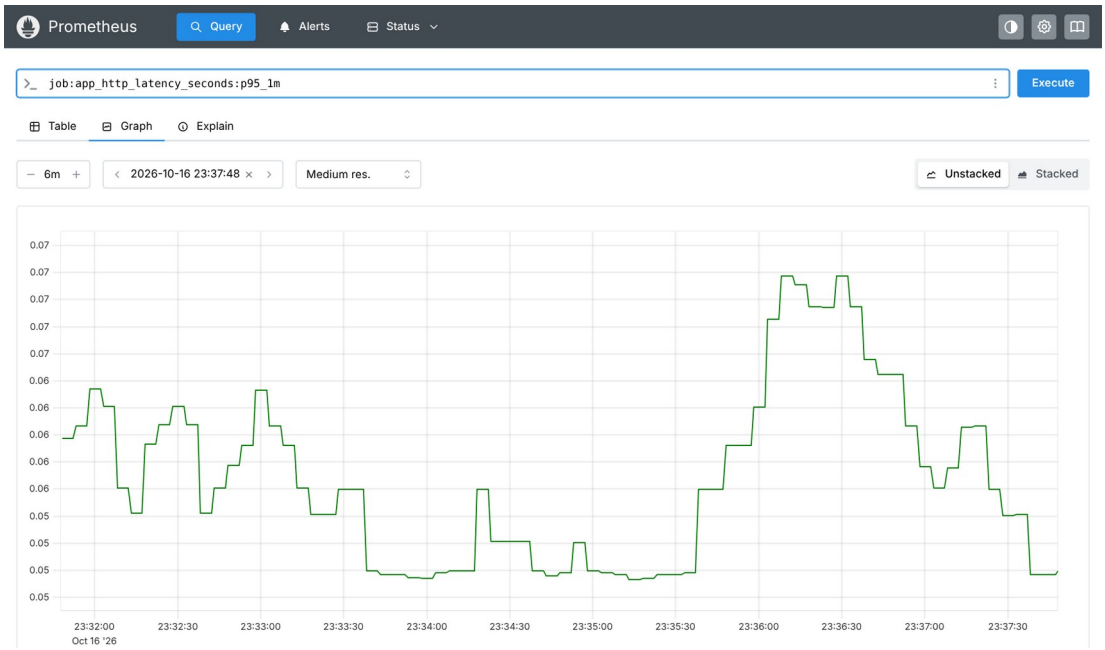


Figura 4. *Latencia p95 (ventana de 1 min) durante la línea base en la interfaz de Prometheus*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: el p95 oscila entre unos 50 y 70 ms sin tendencia, consistente con la latencia normal de la dependencia (p95 de 46,7 ms).

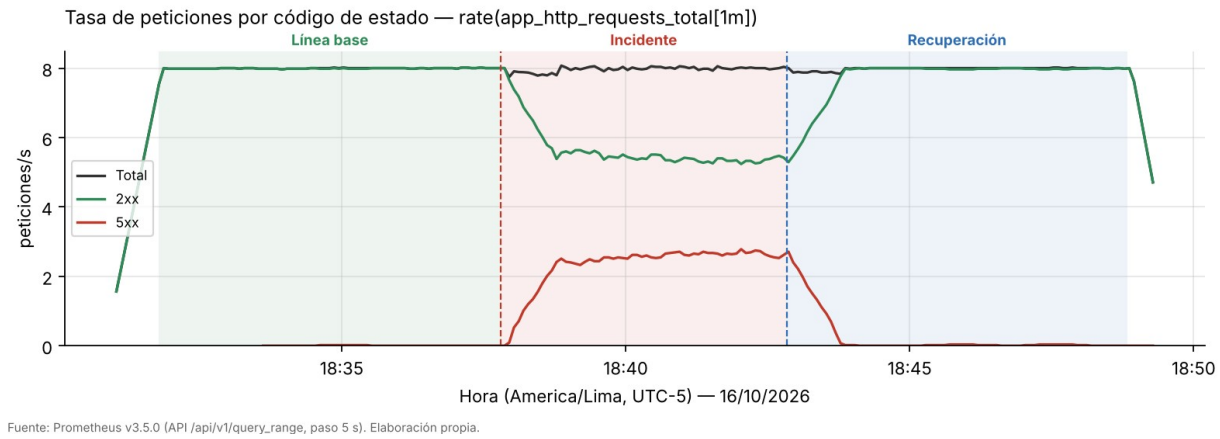
6.3. Incidente

Cambio introducido: a las 18:37:48 se activó el modo de fallo de inventario-db (latencia de 0,6–1,4 s y 30 % de timeouts), sin modificar el tráfico ni los recursos. **Duración:** 302 s. **Recuperación:** a las 18:42:50 se desactivó el fallo. El generador de tráfico registró, desde el lado del cliente, 2 400 peticiones, 778 errores (32,4 %) y latencia media de 1,063 s, coherente con lo medido por Prometheus.

Tabla 6. *Comparación de indicadores por fase (consultas evaluadas sobre toda la ventana de cada fase)*

Indicador (consulta)	Línea base	Incidente	Recuperación
Peticiones por segundo (Q2b)	7,99	8,01	8,00
Errores 5xx (Q3)	0,035 %	32,38 %	0,176 %
Latencia media HTTP (Q4a)	24,1 ms	1 059,9 ms	23,7 ms
Latencia p95 HTTP (Q4b)	54,7 ms	2 195,8 ms	55,9 ms
Máximo del p95 de 1 min (Q5)	71,8 ms	2 263,2 ms	2 265,6 ms *
p95 de inventario-db (Q6)	46,7 ms	1 449,4 ms	46,6 ms
Errores de inventario-db (Q7)	≈ 1	≈ 781	≈ 5
CPU host media / máx. 1 min (Q8a/b)	3,8 % / 7,6 %	2,7 % / 4,1 %	2,6 % / 3,9 %
CPU del proceso app (Q8c)	0,0115 núcleos	0,0116 núcleos	0,0115 núcleos
Memoria disponible mínima (Q9a)	90,1 %	91,6 %	92,0 %
RSS máx. de la app (Q9b)	32,2 MB	33,6 MB	33,9 MB
Concurrencia máxima (Q10)	1	10	1
Utilización máx. de E/S de disco (Q11)	1,11 %	0,04 %	0,04 %
Disponibilidad mínima up (Q1)	1	1	1

Fuente: resultados/resumen_fases.csv. (*) El máximo de la recuperación incluye el primer minuto, cuya ventana rate[1m] todavía contiene muestras del incidente. Los conteos con «≈» provienen de increase(), que extrapola.

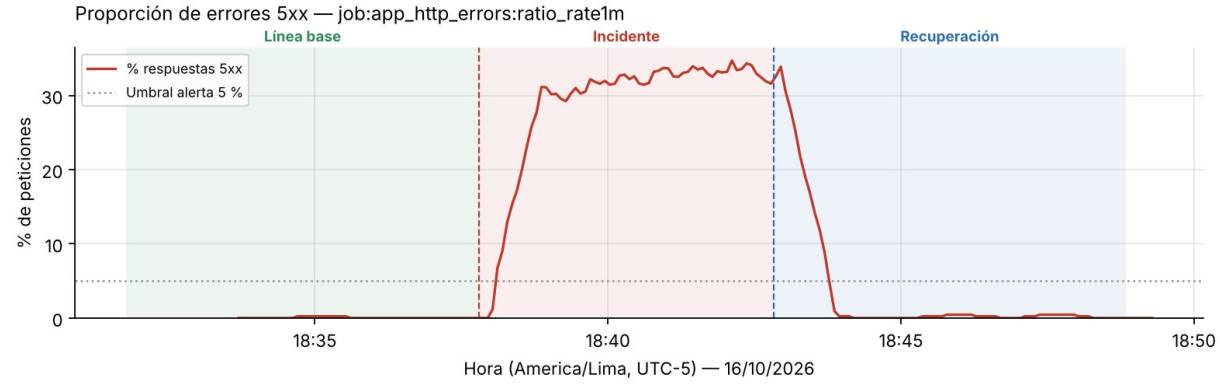


Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 5. *Tasa de peticiones por código de estado*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: el tráfico total se mantuvo en ≈ 8 req/s en las tres fases; durante el incidente una parte de las respuestas pasó de 2xx a 5xx. Que la carga no cambiara descarta un pico de tráfico como causa.

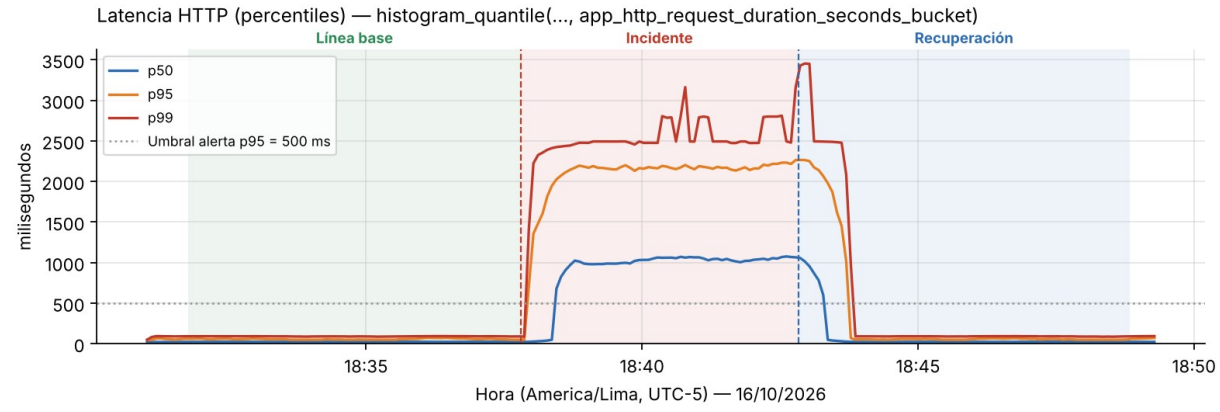


Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 6. *Proporción de respuestas 5xx con el umbral de alerta*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: la proporción subió desde ≈ 0 % hasta estabilizarse en 31–35 % (promedio de la fase 32,4 %) y volvió a la base en unos 60 s tras desactivar el fallo; la rampa de subida y bajada refleja la ventana de 1 minuto de rate().

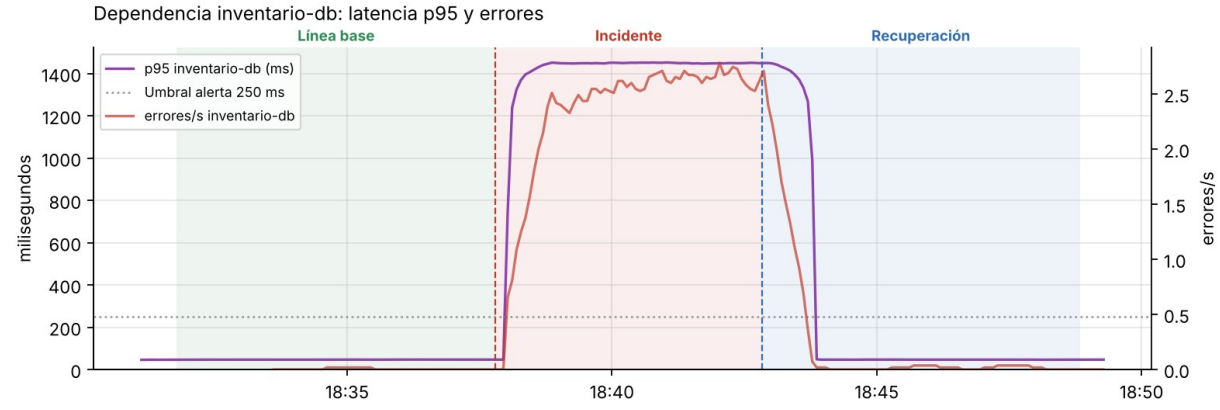


Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 7. *Latencia HTTP: percentiles p50, p95 y p99*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: los tres percentiles saltaron en el mismo intervalo de 5 s en que se activó el fallo; el p95 se mantuvo cerca de 2,2 s. Como los percentiles se estiman por interpolación entre cubetas (1,5 y 2,5 s), su valor es aproximado, pero el cambio de magnitud (≈ 40 veces) es inequívoco.

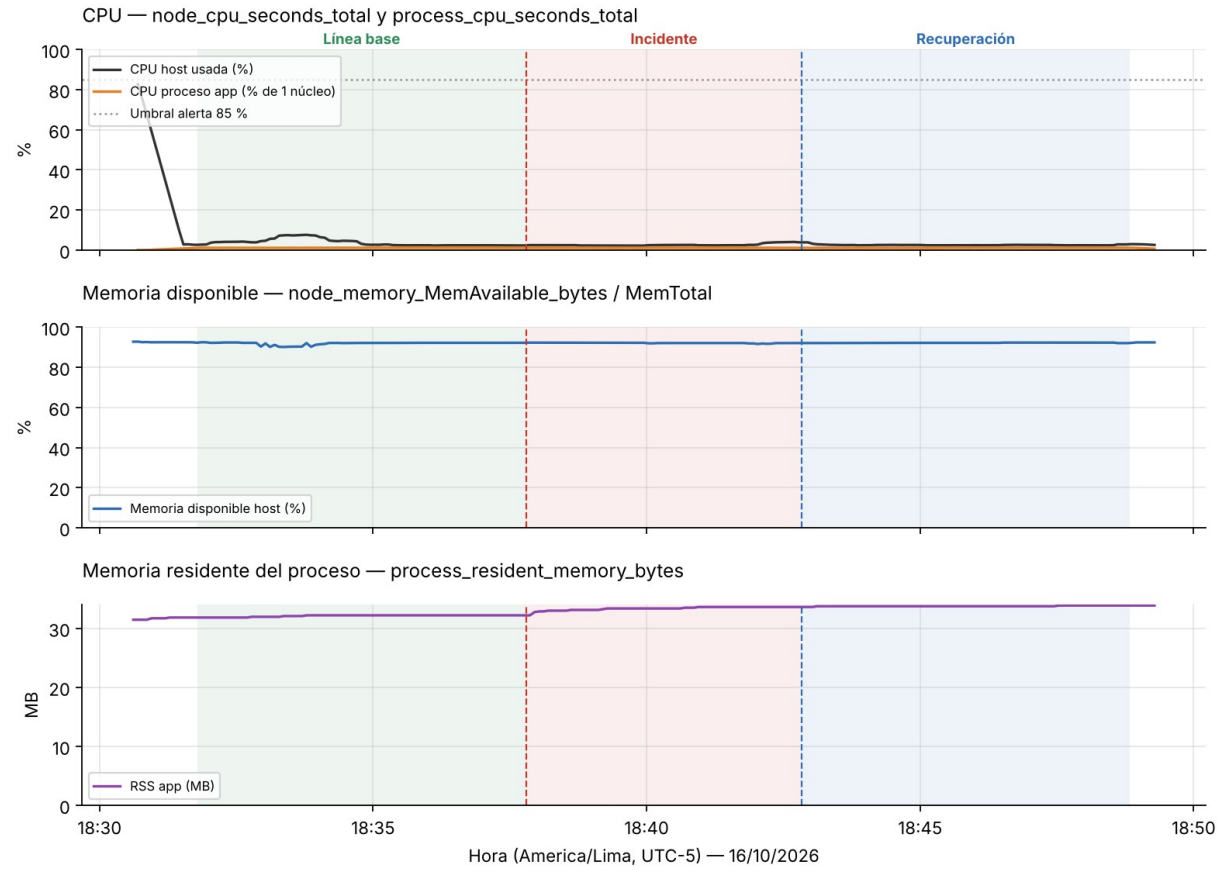


Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 8. *Latencia p95 y errores por segundo de inventario-db*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: el p95 de la dependencia pasó de 46,7 ms a 1 449,4 ms y aparecieron ≈ 2,6 errores/s de timeout. Es la señal más cercana al origen del fallo y la primera en superar su umbral.

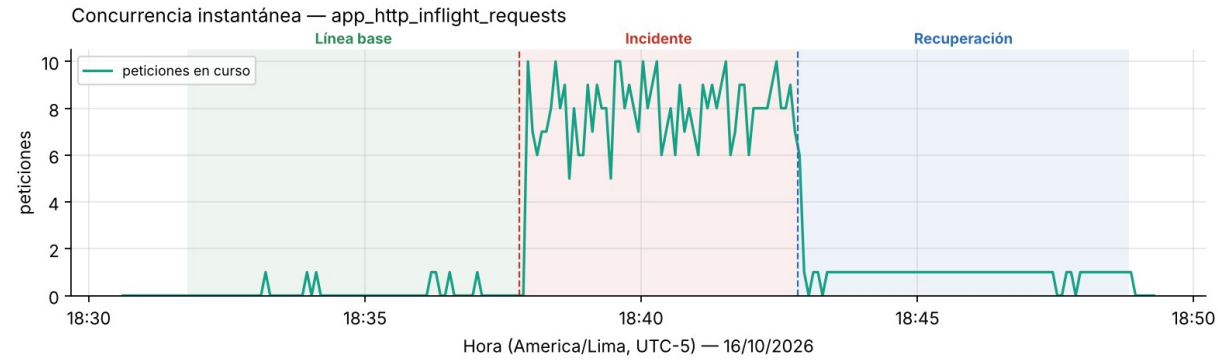


Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 9. *CPU y memoria del host y del proceso de la aplicación*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: la CPU del host promedió 2,7 % durante el incidente (menos que en la línea base) y el proceso de la app consumió lo mismo en las tres fases (≈ 0,012 núcleos). La memoria disponible y el RSS no cambiaron de forma relevante. El pico de CPU antes de las 18:31 corresponde al arranque de los servicios, fuera del experimento.



Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 10. *Peticiones en curso*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: la concurrencia subió de 1 a un máximo de 10. Por la ley de Little, 8 req/s × 1,06 s ≈ 8,5 peticiones simultáneas: el aumento es consecuencia de respuestas más lentas, no de más tráfico.

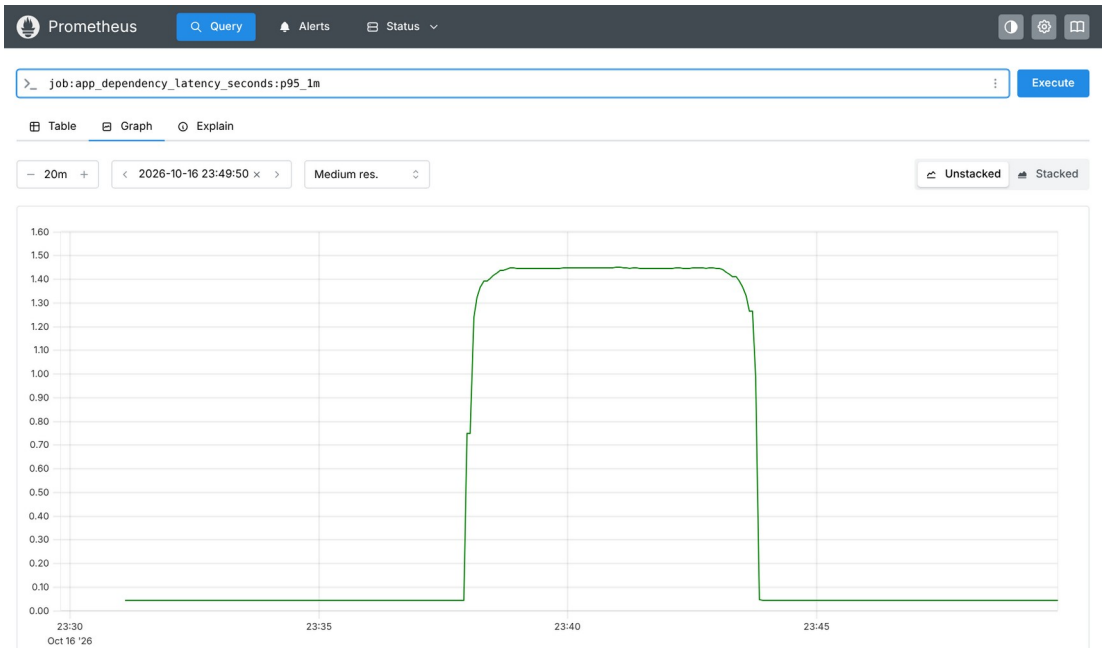


Figura 11. *p95 de inventario-db en la interfaz de Prometheus (regla de registro)*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: la serie job:app_dependency_latency_seconds:p95_1m reproduce el escalón observado en los datos exportados, lo que confirma la trazabilidad entre la TSDB, las consultas y los gráficos.

6.4. Recuperación

Tras desactivar el fallo a las 18:42:50, la latencia de la dependencia volvió a su nivel normal en el siguiente intervalo de recolección, y las señales basadas en ventanas de 1 minuto regresaron a la base en unos 60 s. En la fase de recuperación completa el p95 HTTP fue 55,9 ms (base: 54,7 ms) y los errores 0,18 %, valores equivalentes a la línea base. Los pocos errores restantes se explican por la tasa transitoria normal de la dependencia y por peticiones iniciadas durante el fallo que terminaron después del corte.

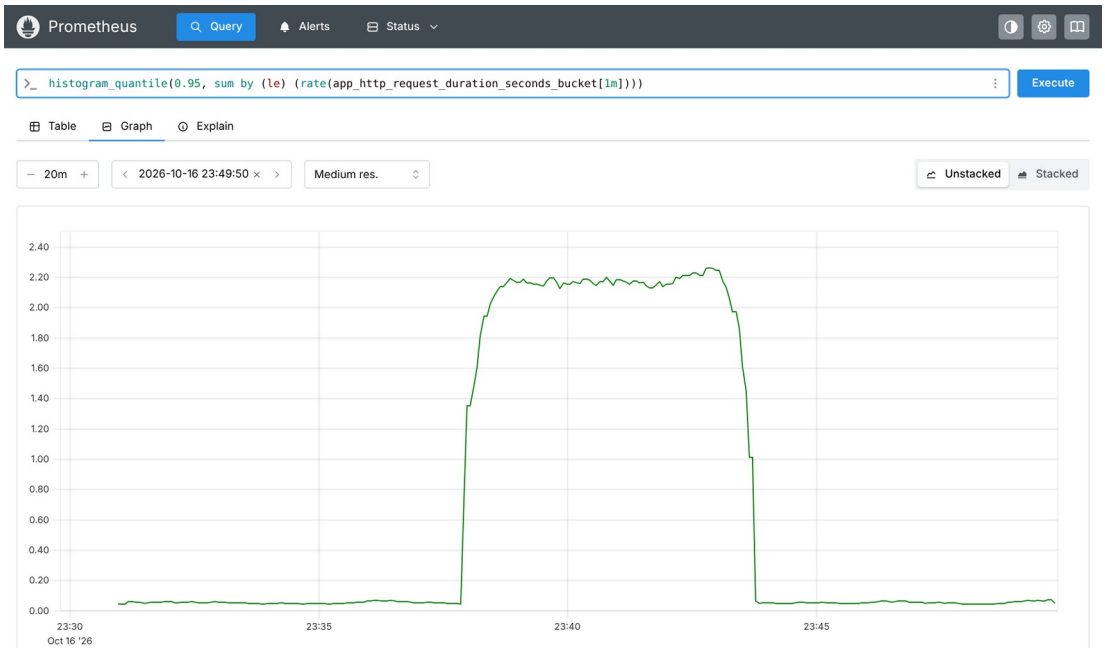


Figura 12. *Ciclo completo de la latencia p95: línea base, incidente y recuperación*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: la captura de la interfaz de Prometheus muestra el escalón de ≈ 0,05 s a ≈ 2,2 s y el retorno completo al nivel inicial, sin degradación residual.

6.5. Consultas PromQL

Cada consulta se evaluó con la API de Prometheus al final de cada fase, usando como ventana de rango la duración completa de la fase (Δ = 361 s, 302 s y 360 s). Los resultados completos están en `resultados/consultas_promql.json` (57 evaluaciones).

Tabla 7. *Consultas PromQL principales, resultados e interpretación*

ID	PromQL	Objetivo	Resultado (base / incidente / recuperación)	Interpretación
Q1	min_over_time(up{job="catalogo-app"}[Δ])	Disponibilidad del objetivo	1 / 1 / 1	El objetivo nunca dejó de ser recolectado.
Q2b	sum(rate(app_http_requests_total[Δ]))	Tasa por segundo (rate)	7,99 / 8,01 / 8,00	La carga fue constante.
Q2c	sum by (endpoint, status) (rate(app_http_requests_total[Δ]))	Tasa por endpoint y estado	503 en incidente: productos 1,65/s; checkout 0,94/s; raíz 0	Solo fallan los endpoints que usan inventario-db.
Q3	sum(increase(...{status=~"5.."}[Δ])) / sum(increase(...[Δ]))	Tasa de errores	0,035 % / 32,38 % / 0,18 %	Unas 900 veces más errores en el incidente.
Q4b	histogram_quantile(0.95, sum by (le) (rate(..._bucket[Δ])))	Latencia p95	54,7 ms / 2 195,8 ms / 55,9 ms	Degradación severa y reversible.
Q4c	histogram_quantile(0.95, sum by (endpoint, le) (rate(..._bucket[Δ])))	Agrupación by endpoint	raíz 9,5 ms en las 3 fases; checkout 91 ms → 2 490 ms	La ruta sin dependencia no se degradó; checkout (2 llamadas) es la más afectada.
Q5	max_over_time(job:app_http_latency_seconds:p95_1m[Δ])	Máximo temporal	71,8 ms / 2 263,2 ms	Pico del incidente ≈ 31 veces el pico base.
Q6	histogram_quantile(0.95, sum by (le) (rate(app_dependency_duration_seconds_bucket[Δ])))	Latencia de dependencia	46,7 ms / 1 449,4 ms / 46,6 ms	Origen de la latencia observada.
Q8a	1 - avg by (instance)	Uso de CPU (avg by)	3,8 % / 2,7 % / 2,6 %	Sin presión de CPU.

ID	PromQL	Objetivo	Resultado (base / incidente / recuperación)	Interpretación
	(rate(node_cpu_seconds_total{mode="idle"}[Δ]))			
Q9a	min_over_time((MemAvailable / MemTotal)[Δ:5s])	Uso de memoria (subconsulta)	90,1 % / 91,6 % / 92,0 %	Sin presión de memoria.

Fuente: resultados/consultas_promql.json. «…» abrevia el nombre de métrica indicado en el Anexo A.

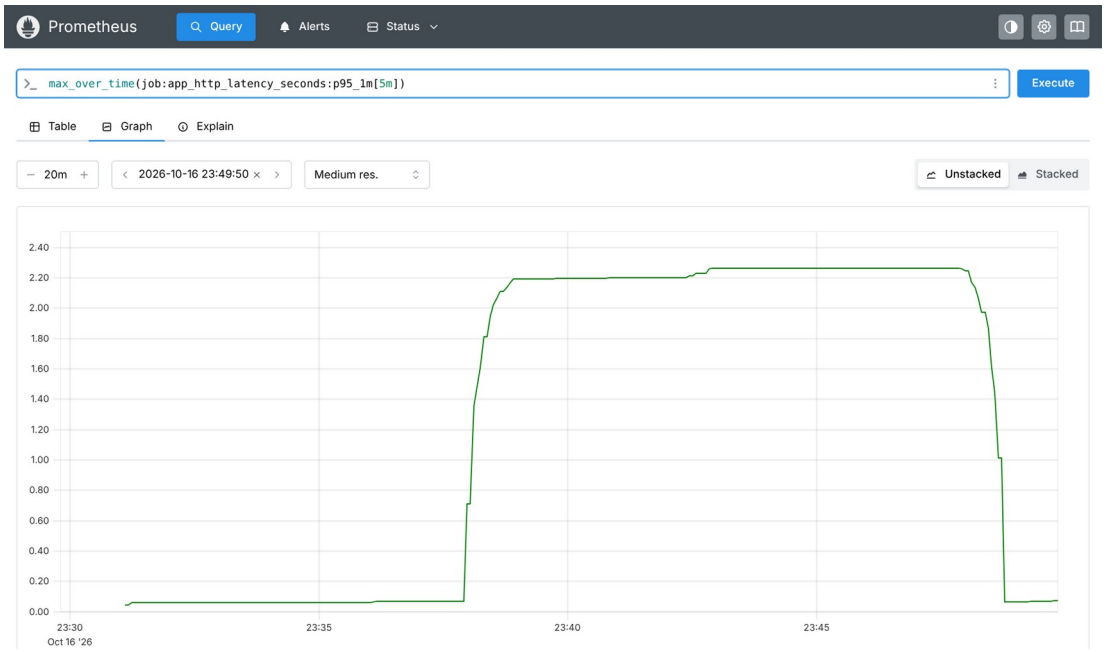


Figura 13. *Consulta max_over_time sobre el p95 en la interfaz de Prometheus*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: max_over_time con ventana de 5 min conserva el pico durante 5 minutos después del incidente; es útil para revisar periodos pasados sin perder máximos breves, pero no debe usarse para detectar la recuperación en tiempo real.

6.6. Alertas

Tabla 8. *Transiciones de las alertas durante el experimento (hora de Lima)*

Alerta	Condición (for)	Pending desde	Firing desde	Firing hasta	Resultado
DependenciaLenta	p95 dependencia > 250 ms (1 m)	18:38:02	18:39:02	18:43:47	Disparó
LatenciaP95Alta	p95 HTTP > 500 ms (1 m)	18:38:02	18:39:02	18:43:47	Disparó
TasaErroresAlta	5xx > 5 % (1 m)	18:38:07	18:39:07	18:43:42	Disparó
CPUHostAlta	CPU host > 85 % (2 m)	—	—	—	No disparó (correcto)
MemoriaHostBaja	Mem. disponible < 10 % (2 m)	—	—	—	No disparó (correcto)
ObjetivoCaido	up == 0 (30 s)	—	—	—	No disparó (correcto)

Fuente: resultados/alertas_transiciones.json (serie ALERTS de Prometheus, paso 5 s).

Las alertas pasaron a *pending* 14–19 s después de activar el fallo y a *firing* unos 74–79 s después (por la cláusula `for: 1m`). Se resolvieron entre 52 y 57 s después de desactivar el fallo, cuando las ventanas de 1 minuto dejaron de contener muestras del incidente. Las alertas de recursos no se activaron, lo que coincide con los datos de CPU y memoria.

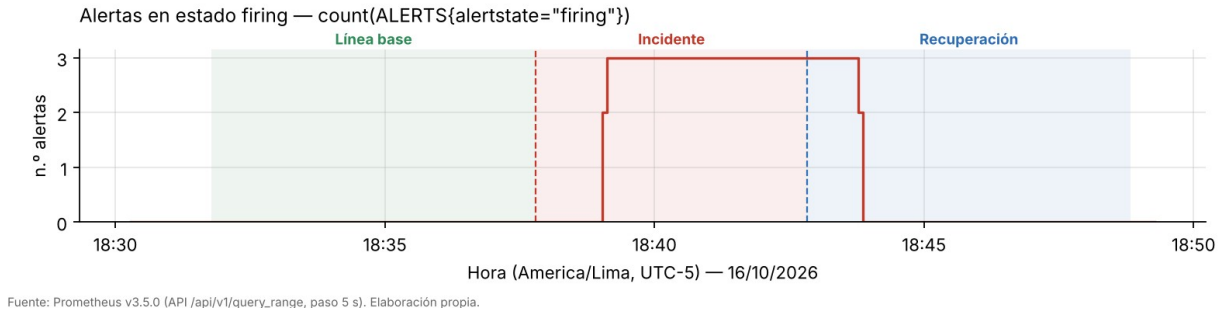


Figura 14. *Número de alertas en estado firing*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: las tres alertas de servicio estuvieron activas solo durante el incidente (más el desfase de las ventanas); fuera de él el conteo es 0, es decir, no hubo falsos positivos en la línea base ni en la recuperación.

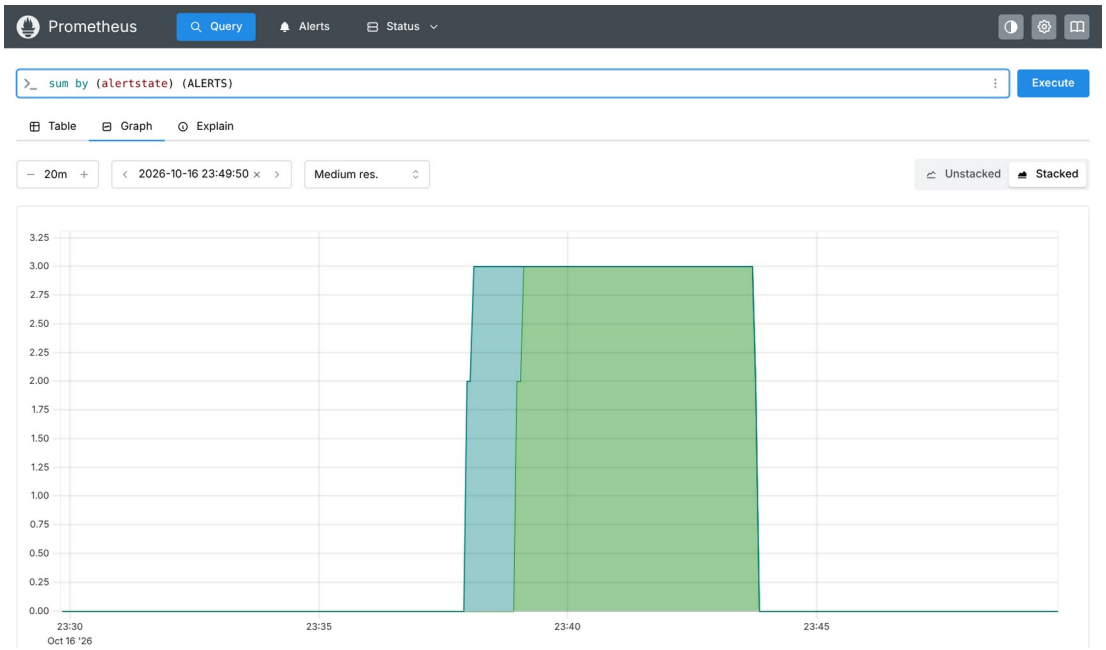


Figura 15. *Alertas por estado: sum by (alertstate) (ALERTS), gráfico apilado*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: (azul: pending; verde: firing). Unos segundos después del inicio aparecen alertas en pending y, un minuto después, las tres pasan a firing; desaparecen cerca de un minuto después de retirar el fallo. Verifica que las reglas se evaluaron en el entorno real.

7. Análisis del fallo

7.1. Línea temporal

Tabla 9. *Línea temporal del incidente*

Hora (Lima)	Evento u observación	Fuente
18:31:47	Inicio de la línea base; señales estables	timeline.csv
18:37:48	Se activa el fallo de inventario-db (cambio declarado)	timeline.csv
18:37:48–18:37:53	p95 de la dependencia y p95 HTTP aumentan en el mismo intervalo de recolección	Figuras 7, 8 y 16
18:38:02	LatenciaP95Alta y DependenciaLenta en pending	ALERTS
≈ 18:38:50	La proporción de 5xx se estabiliza en	Figura 6

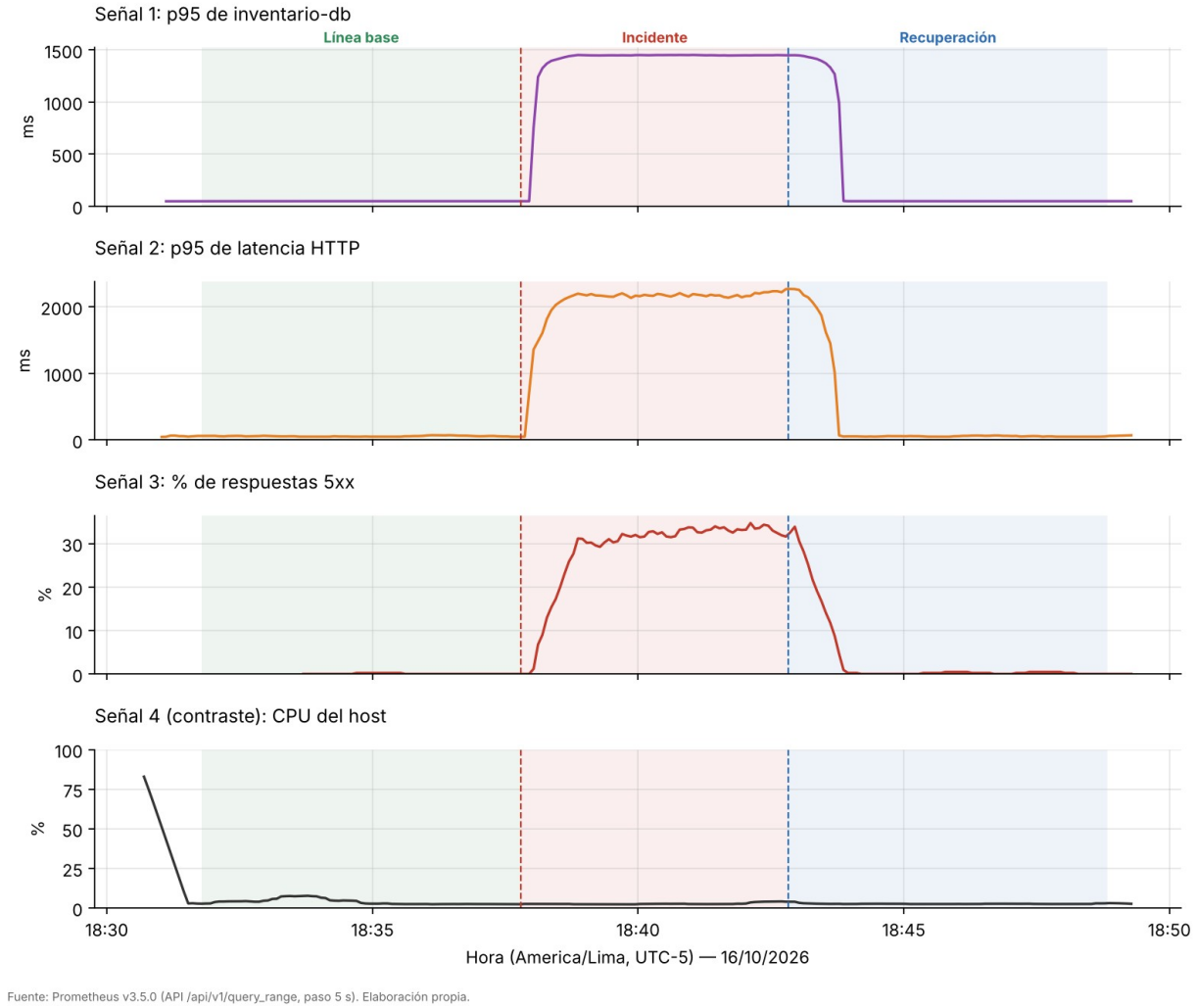
Hora (Lima)	Evento u observación	Fuente
	31–35 % (fin de la rampa de 1 min)	
18:39:02	DependenciaLenta y LatenciaP95Alta en firing	ALERTS
18:39:07	TasaErroresAlta en firing	ALERTS
18:42:50	Se desactiva el fallo; la latencia de la dependencia se normaliza	timeline.csv
18:43:42–18:43:47	Se resuelven las tres alertas	ALERTS
18:48:50	Fin de la recuperación; indicadores iguales a la línea base	Tabla 6

7.2. Hoja de análisis del incidente

Tabla 10. *Hoja de análisis del incidente (formato de la guía)*

Elemento	Registro del estudiante
Intervalo analizado	16/10/2026, 18:31:47 a 18:48:50 (Lima); incidente 18:37:48–18:42:50 (302 s).
Síntoma observable	Respuestas lentas e intermitentes: p95 de 54,7 ms a 2,20 s y 32,4 % de respuestas 503, con tráfico constante y servicio alcanzable (up = 1).
Métrica 1 y hallazgo	`app_dependency_duration_seconds` (p95): de 46,7 ms a 1 449,4 ms (≈ 31×), con ≈ 781 errores de timeout de inventario-db.
Métrica 2 y hallazgo	`app_http_request_duration_seconds` (p95): de 54,7 ms a 2 195,8 ms; por endpoint, la raíz (sin dependencia) se mantuvo en 9,5 ms y checkout (dos llamadas) llegó a 2,49 s.
Métrica 3 y hallazgo	`app_http_requests_total` (ratio 5xx): de 0,035 % a 32,4 %; los 503 aparecen solo en /api/productos y /api/checkout.
Métricas de contraste	CPU del host 2,7 % (máx. 4,1 %), CPU de la app invariable, memoria disponible ≥ 91,6 %, E/S de disco ≈ 0 %.
Relación temporal entre señales	La latencia de la dependencia y la latencia HTTP cambian en el mismo intervalo de 5 s; los errores crecen en rampa durante el minuto siguiente (efecto de la ventana de rate). Las tres señales vuelven a la base al retirar el fallo. CPU y memoria no cambian en ningún momento.
Hipótesis más consistente	Fallo de la dependencia inventario-db (latencia alta y timeouts) que se propaga a los endpoints que la consultan.
Evidencia que contradice alternativas	CPU: el uso del host bajó (3,8 % → 2,7 %) y el proceso no varió. Memoria: sin descenso. E/S: utilización máxima 0,04 %. Pico de tráfico: la tasa fue 8 req/s en todas las fases. Fallo general del host: la ruta raíz no se degradó.
Acción inmediata propuesta	Aislar la dependencia: timeout de 300 ms y cortacircuitos con respuesta degradada (catálogo en caché) y revisar o conmutar inventario-db. No escalar CPU ni memoria.
Medida preventiva propuesta	Alertar sobre la latencia y los errores de la dependencia, definir un SLO con alertas por tasa de consumo del presupuesto de errores y añadir trazas distribuidas.

7.3. Correlación de señales y contraste de hipótesis



Fuente: Prometheus v3.5.0 (API /api/v1/query_range, paso 5 s). Elaboración propia.

Figura 16. *Correlación temporal de tres señales del servicio y una señal de contraste*

Fuente: elaboración propia con datos reales de Prometheus 3.5.0 exportados del laboratorio.

Interpretación: las señales 1 a 3 cambian juntas al inicio y al final del fallo, con la dependencia como primera en reaccionar; la señal 4 (CPU del host) permanece plana. Hechos observados: coincidencia temporal y selectividad por endpoint. Inferencia: la dependencia es la causa más consistente.

Tabla 11. *Contraste de hipótesis con la evidencia*

Hipótesis de la guía	Métricas revisadas	Criterio de contraste	Resultado
Saturación de CPU	node_cpu_seconds_total, process_cpu_seconds_total, latencia	Coincidencia entre presión de CPU y degradación	**Descartada.** CPU host 2,7 % (base 3,8 %); proceso sin cambios.
Presión de memoria	node_memory_MemAvailable_bytes, process_resident_memory_bytes	Descenso sostenido cerca del incidente	**Descartada.** Disponible ≥ 91,6 %; RSS +1,4 MB.
Problema de E/S o almacenamiento	node_disk_io_time_seconds_total	Cambio anómalo coincidente	**Descartada.** Utilización máxima 0,04 %.
Fallo de servicio o dependencia	up, errores, latencia, latencia/errores de inventario-db, concurrencia	Cambio correlacionado en el servicio	**Confirmada.** up = 1, pero la dependencia aumenta su p95 31× y concentra los errores.

7.4. Diagnóstico argumentado

Se distinguen tres niveles. **Hechos observados:** el servicio siguió disponible (up = 1) y recibió la misma carga, pero su latencia y su proporción de errores aumentaron en el mismo intervalo en que lo hizo la latencia de inventario-db; la ruta que no usa la dependencia no cambió; los recursos del host no variaron. **Inferencia:** la degradación se origina en la dependencia y se propaga a los endpoints que la consultan; checkout, que hace dos llamadas, sufre más. El aumento de concurrencia es un efecto (ley de Little), no una causa. **Contexto:** el cambio declarado del experimento fue precisamente la degradación de inventario-db, por lo que el diagnóstico coincide con la causa real. El marcador `app_fault_injection_active` no se usó como evidencia, para que el razonamiento dependa solo de las señales que estarían disponibles en un incidente real.

La sospecha inicial de la guía, «presión de recursos», no se confirmó: este caso muestra que una gráfica de latencia por sí sola no identifica la causa, y que escalar CPU o memoria no habría resuelto el problema.

7.5. Limitaciones del análisis

- La dependencia es simulada dentro del proceso de la app; en un sistema real habría red, base de datos y conexiones con métricas propias.
- Los percentiles se estiman por interpolación lineal dentro de las cubetas del histograma; los valores de p95 son aproximados.
- Las ventanas de 1 minuto de rate() suavizan y retrasan las señales hasta unos 60 s; los errores muestran una rampa que no existió en la realidad.
- El experimento se ejecutó una vez, en un solo host y durante cinco minutos; no se evaluaron variaciones de carga ni fallos parciales.
- node_exporter mide la VM compartida del entorno. Grafana no pudo ejecutarse en ese entorno, por lo que no se incluyen capturas de sus paneles.
- increase() extrapola en los bordes de la ventana, por lo que los conteos por fase son aproximados.

8. Propuesta de mejora

8.1. Acción correctiva inmediata

- Aplicar un timeout de 300 ms a cada llamada a inventario-db (≈ 6 veces su p95 normal de 47 ms) para limitar la latencia que se propaga.
- Activar un cortacircuitos (*circuit breaker*) que, ante más de un 20 % de fallos en 30 s, responda con datos en caché o una respuesta degradada en lugar de 503.
- Revisar inventario-db (conexiones, consultas lentas, despliegues recientes) y, si existe, conmutar a una réplica sana.
- No escalar CPU ni memoria de la app: los datos muestran que no son el cuello de botella.

8.2. Medidas preventivas

- Definir un SLO, por ejemplo 99 % de respuestas exitosas y p95 < 300 ms en 30 días, y alertar por tasa de consumo del presupuesto de errores (Beyer et al., 2016).
- Reintentos con espera exponencial y aleatoriedad (*jitter*) solo para operaciones idempotentes, con límite de reintentos para no amplificar la carga.

- Aislar recursos por dependencia (*bulkhead*): un grupo de hilos dedicado evita que una dependencia lenta agote la capacidad del servicio.
- Añadir trazas distribuidas y métricas propias de la base de datos para reducir la incertidumbre en futuros diagnósticos.
- Probar periódicamente este escenario (prueba de caos controlada) con los scripts del repositorio.

8.3. Reglas de alerta y umbrales justificados

Tabla 12. *Reglas de alerta implementadas y propuestas*

Alerta	Condición	Justificación del umbral
DependenciaLenta	p95 inventario-db > 250 ms durante 1 m	≈ 5× el p95 base (46,7 ms); es la señal más cercana a la causa.
LatenciaP95Alta	p95 HTTP > 500 ms durante 1 m	≈ 9× el p95 base (54,7 ms); mide el impacto en el usuario.
TasaErroresAlta	5xx > 5 % durante 1 m	La base es < 0,2 %; 5 % evita falsos positivos por errores transitorios.
CPUHostAlta / MemoriaHostBaja	CPU > 85 % 2 m / memoria < 10 % 2 m	Detectan presión de recursos; sirven para descartarla rápidamente.
ConsumoPresupuestoRapido (propuesta)	ratio 5xx 5 m > 14,4 × 1 % y ratio 1 h > 14,4 × 1 %	Alerta multiventana por tasa de consumo del SLO del 99 %.

```
- alert: ConsumoPresupuestoRapido      # propuesta para el SLO de 99 % de éxito
  expr: |
    ( sum(rate(app_http_requests_total{status=~"5.."}[5m])) /
sum(rate(app_http_requests_total[5m])) > 14.4 * 0.01 )
    and
    ( sum(rate(app_http_requests_total{status=~"5.."}[1h])) /
sum(rate(app_http_requests_total[1h])) > 14.4 * 0.01 )
  for: 2m
  labels: {severity: critical}
```

8.4. Métricas de seguimiento y verificación de la recuperación

Se seguirán `job:app_dependency_latency_seconds:p95_1m`, `job:app_http_latency_seconds:p95_1m`, `job:app_http_errors:ratio_rate1m`, `app_http_inflight_requests` y la serie `ALERTS`. La recuperación se considera verificada cuando, durante 5 minutos consecutivos: (1) el ratio de 5xx es < 1 %; (2) el p95 HTTP es < 100 ms; (3) el p95 de la dependencia es < 100 ms; (4) la concurrencia vuelve a 0–2; y (5) no hay alertas en firing. En este laboratorio los cinco criterios se cumplieron en la fase de recuperación (Tabla 6 y Figura 12).

9. Conclusiones

1. La arquitectura Prometheus/TSDB + PromQL + reglas de alerta permitió observar el incidente con resolución de 5 s y trazar cada conclusión hasta una consulta concreta: tres alertas de servicio dispararon unos 75 s después del inicio y se resolvieron menos de un minuto después de la corrección, sin falsos positivos en las fases normales (Tabla 8).
2. La correlación de tres señales identificó la causa: la latencia de inventario-db subió ≈ 31 veces en el mismo intervalo que la latencia HTTP (p95 de 54,7 ms a 2,20 s) y los errores (32,4 %), mientras la ruta sin dependencia no cambió (Tabla 7 y Figura 16).
3. La hipótesis de presión de recursos fue descartada con datos: CPU del host de 2,7 %, memoria disponible ≥ 91,6 % y E/S casi nula durante el incidente. Escalar recursos no habría resuelto el problema.

4. Las métricas de dependencias y los percentiles de latencia son más útiles que las de infraestructura para detectar este tipo de fallo; por ello se recomienda alertar sobre la dependencia y proteger el servicio con timeouts y cortacircuitos.

10. Referencias bibliográficas

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media. https://sre.google/sre-book/monitoring-distributed-systems/

Docker Inc. (2026). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/

GitHub. (2026). *GitHub Codespaces overview*. GitHub Docs. https://docs.github.com/en/codespaces/overview

Grafana Labs. (2026a). *Prometheus data source*. Grafana documentation. https://grafana.com/docs/grafana/latest/datasources/prometheus/

Grafana Labs. (2026b). *Provision Grafana*. Grafana documentation. https://grafana.com/docs/grafana/latest/administration/provisioning/

Majors, C., Fong-Jones, L., & Miranda, G. (2022). *Observability engineering: Achieving production excellence*. O'Reilly Media.

Prometheus Authors. (2026a). *Overview*. Prometheus documentation. https://prometheus.io/docs/introduction/overview/

Prometheus Authors. (2026b). *Data model*. Prometheus documentation. https://prometheus.io/docs/concepts/data_model/

Prometheus Authors. (2026c). *Querying Prometheus*. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/querying/basics/

Prometheus Authors. (2026d). *Storage*. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/storage/

Prometheus Authors. (2026e). *Alerting rules*. Prometheus documentation. https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Prometheus Authors. (2026f). *Histograms and summaries*. Prometheus documentation. https://prometheus.io/docs/practices/histograms/

Prometheus Authors. (2026g). *Node exporter* [Software]. GitHub. https://github.com/prometheus/node_exporter

11. Cierre y metacognición

1. ¿Qué métrica fue más útil para detectar o explicar el incidente y por qué?

La latencia p95 de la dependencia (`app_dependency_duration_seconds`). La latencia HTTP y los errores mostraron que el servicio estaba degradado, pero no por qué. La latencia de inventario-db cambió primero, en el mismo intervalo de 5 s, y en una magnitud (≈ 31×) que explica por sí sola la latencia de los endpoints que la consultan. Además, su alerta fue la más específica: apunta a la causa y no solo al síntoma.

2. ¿Qué hipótesis inicial fue confirmada o descartada por los datos?

La hipótesis inicial sugerida por el caso, presión de recursos (CPU o memoria), quedó descartada: la CPU del host bajó de 3,8 % a 2,7 %, el proceso de la app consumió lo mismo y la memoria disponible no bajó del 91 %. Se confirmó la hipótesis de fallo de una dependencia, sustentada por la coincidencia temporal y porque la ruta que no la usa no se degradó.

3. ¿Qué información adicional habría reducido la incertidumbre del diagnóstico?

Trazas distribuidas que muestren el tiempo de cada petición dentro de inventario-db, métricas propias de la base de datos (conexiones, consultas lentas, bloqueos) y un registro de cambios o despliegues. También ayudarían histogramas nativos o cubetas más finas para estimar los percentiles con más precisión y varias repeticiones del experimento para medir la variabilidad.

4. ¿Cómo automatizaría la detección temprana de un evento similar?

Con las reglas ya probadas (DependenciaLenta, LatenciaP95Alta, TasaErroresAlta) enviadas mediante Alertmanager a un canal del equipo, más una alerta por tasa de consumo del SLO. Se añadirían sondas sintéticas periódicas contra los endpoints críticos, un runbook enlazado en cada alerta que indique revisar primero la dependencia y una prueba de caos programada para comprobar que las alertas siguen funcionando.

12. Autoevaluación frente a la rúbrica

Tabla 13. *Autoevaluación (la calificación corresponde al docente)*

Criterio (puntaje)	Evidencia que sustenta el cumplimiento
Explicación del monitoreo y TSDB (4)	Sección 3 (Tabla 2, con referencias), Sección 4 (Figura 1 y Tabla 3), configuración de TSDB y recolección.
Obtención y presentación de métricas (4)	Inventario (Tabla 5), datos reales exportados con periodos y unidades (Tabla 6), 10 consultas interpretadas (Tabla 7), capturas reales.
Análisis del fallo (6)	Línea temporal (Tabla 9), hoja de análisis (Tabla 10), correlación de 3 señales más contraste (Figura 16), 4 hipótesis contrastadas (Tabla 11), diagnóstico y limitaciones.
Propuesta de respuesta y prevención (3)	Sección 8: acción inmediata, prevención, reglas con umbrales justificados por la línea base y procedimiento de verificación.
Comunicación académica y reproducibilidad (3)	Estructura de la guía, figuras y tablas numeradas con fuente, APA 7, repositorio con scripts y datos.

Pendiente declarado: capturas de los paneles de Grafana, que se obtienen al ejecutar `docker compose up` en GitHub Codespaces.

13. Anexos

Anexo A. Consultas PromQL completas

```
# Q1 — Disponibilidad del objetivo: mínimo de up en la fase
min_over_time(up{job="catalogo-app"}[Δ])

# Q2a — Peticiones atendidas en la fase (incremento del contador)
sum(increase(app_http_requests_total[Δ]))

# Q2b — Tasa media de peticiones por segundo en la fase
sum(rate(app_http_requests_total[Δ]))

# Q2c — Tasa por endpoint y código de estado
sum by (endpoint, status) (rate(app_http_requests_total[Δ]))

# Q3 — Proporción de respuestas 5xx en la fase
sum(increase(app_http_requests_total{status=~"5.."}[Δ])) / sum(increase(app_http_requests_total[Δ]))
```

```
# Q4a — Latencia media HTTP en la fase
sum(rate(app_http_request_duration_seconds_sum[Δ])) /
sum(rate(app_http_request_duration_seconds_count[Δ]))

# Q4b — Percentil 95 de latencia HTTP en toda la fase
histogram_quantile(0.95, sum by (le) (rate(app_http_request_duration_seconds_bucket[Δ])))

# Q4c — p95 de latencia por endpoint (agrupación by)
histogram_quantile(0.95, sum by (endpoint, le) (rate(app_http_request_duration_seconds_bucket[Δ])))

# Q5 — Máximo temporal del p95 (ventana 1 m) dentro de la fase
max_over_time(job:app_http_latency_seconds:p95_1m[Δ])

# Q6 — p95 de latencia de inventario-db en la fase
histogram_quantile(0.95, sum by (le) (rate(app_dependency_duration_seconds_bucket[Δ])))

# Q7 — Errores de la dependencia (incremento) por motivo
sum(increase(app_dependency_errors_total[Δ])) or vector(0)

# Q8a — Uso medio de CPU del host (1 - idle), avg por instancia
1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[Δ]))

# Q8b — Máximo del uso de CPU del host (ventanas de 1 m) en la fase
max_over_time((1 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m])))[Δ:5s])

# Q8c — CPU consumida por el proceso de la app (núcleos)
rate(process_cpu_seconds_total{job="catalogo-app"}[Δ])

# Q9a — Mínimo de memoria disponible del host (ratio)
min_over_time((node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)[Δ:5s])

# Q9b — Máximo de memoria residente de la app (MB)
max_over_time(process_resident_memory_bytes{job="catalogo-app"}[Δ]) / 1024 / 1024

# Q10 — Concurrencia máxima (peticiones en curso)
max_over_time(app_http_inflight_requests[Δ])

# Q11 — Máxima utilización de E/S de disco (fracción de tiempo ocupado)
max_over_time(max(rate(node_disk_io_time_seconds_total[1m]))[Δ:5s])

# Q12 — Alertas en estado firing durante la fase
max_over_time(ALERTS{alertstate="firing"}[Δ])
```

Δ es la duración de cada fase (361 s, 302 s y 360 s). Consultas de los paneles (ventana móvil de 1 minuto):

```
sum by (endpoint, status) (rate(app_http_requests_total[1m]))
job:app_http_errors:ratio_rate1m
histogram_quantile(0.95, sum by (le) (rate(app_http_request_duration_seconds_bucket[1m])))
job:app_dependency_latency_seconds:p95_1m
1 - avg(rate(node_cpu_seconds_total{mode="idle"}[1m]))
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
ALERTS{alertstate="firing"}
```

Anexo B. Reglas de registro y de alerta (prometheus/alerts.yml)

```
groups:
  - name: sli-catalogo
    interval: 5s
    rules:
      # Reglas de registro (SLI) reutilizadas por alertas y dashboards
      - record: job:app_http_requests:rate1m
        expr: sum by (job) (rate(app_http_requests_total{endpoint!="/metrics"}[1m]))
      - record: job:app_http_errors:ratio_rate1m
        expr: |
          sum by (job) (rate(app_http_requests_total{status=~"5.."}[1m]))
          /
          sum by (job) (rate(app_http_requests_total{endpoint!="/metrics"}[1m]))
      - record: job:app_http_latency_seconds:p95_1m
        expr: histogram_quantile(0.95, sum by (job, le)
(rate(app_http_request_duration_seconds_bucket[1m])))
      - record: job:app_dependency_latency_seconds:p95_1m
        expr: histogram_quantile(0.95, sum by (job, dependency, le)
(rate(app_dependency_duration_seconds_bucket[1m])))
```

```yaml
  - name: alertas-catalogo
    rules:
      - alert: ObjetivoCaido
        expr: up == 0
        for: 30s
        labels:
          severity: critical
        annotations:
          summary: "Prometheus no puede recolectar {{ $labels.job }} ({{ $labels.instance }})"

      - alert: TasaErroresAlta
        expr: job:app_http_errors:ratio_rate1m > 0.05
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Más del 5 % de respuestas 5xx en {{ $labels.job }}"
          description: "Ratio actual: {{ $value | humanizePercentage }}"

      - alert: LatenciaP95Alta
        expr: job:app_http_latency_seconds:p95_1m > 0.5
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "Latencia p95 > 500 ms en {{ $labels.job }}"
          description: "p95 actual: {{ $value | humanizeDuration }}"

      - alert: DependenciaLenta
        expr: job:app_dependency_latency_seconds:p95_1m > 0.25
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "p95 de {{ $labels.dependency }} > 250 ms"
          description: "Posible degradación de dependencia; revisar antes de escalar recursos."

      - alert: CPUHostAlta
        expr: 1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m])) > 0.85
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "CPU del host > 85 % durante 2 minutos"

      - alert: MemoriaHostBaja
        expr: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes < 0.10
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Memoria disponible del host < 10 %"
```

Anexo C. Fragmento de instrumentación y modo de fallo (application/app.py)

```python
def call_dependency():
    """Simula la consulta a la dependencia. Lanza RuntimeError si falla."""
    with _lock:
        fault = _state["fault"]
    start = time.perf_counter()
    try:
        if fault:
            # Degradación: latencia alta y 30 % de timeouts
            time.sleep(random.uniform(0.6, 1.4))
            if random.random() < 0.30:
                DEP_ERRORS.labels(DEPENDENCY, "timeout").inc()
                raise RuntimeError("timeout en inventario-db")
        else:
            time.sleep(random.uniform(0.008, 0.035))
            if random.random() < 0.002:
                DEP_ERRORS.labels(DEPENDENCY, "transient").inc()
                raise RuntimeError("error transitorio")
    finally:
        DEP_LATENCY.labels(DEPENDENCY).observe(time.perf_counter() - start)
```

Anexo D. Scripts y datos exportados

Tabla 14. *Archivos del repositorio*

Archivo	Contenido
scripts/validar_entorno.sh	Verifica herramientas, servicios, objetivos, frescura de muestras, reglas y Grafana
scripts/ejecutar_baseline.sh / _incidente.sh / _recuperacion.sh	Fases del experimento con marcas de tiempo
scripts/loadgen.py	Generador de tráfico controlado (solo contra la app local)
scripts/exportar_metricas.py	Exporta series, consultas por fase y transiciones de alertas
scripts/graficar.py / capturas_prometheus.py	Gráficos con datos reales y capturas de la interfaz
resultados/series_experimento.csv	229 instantes × 21 señales (paso 5 s)
resultados/consultas_promql.json	57 evaluaciones PromQL con objetivo, periodo y resultado
resultados/resumen_fases.csv, timeline.csv	Indicadores por fase y marcas de tiempo
resultados/cliente_*.csv	Vista del cliente: cada petición con estado y latencia
grafana/dashboards/catalogo-observabilidad.json	Dashboard provisionado de 12 paneles

Repositorio: https://github.com/ThiagoIcochea/LaboratorioSemana10

Anexo E. Evidencias complementarias

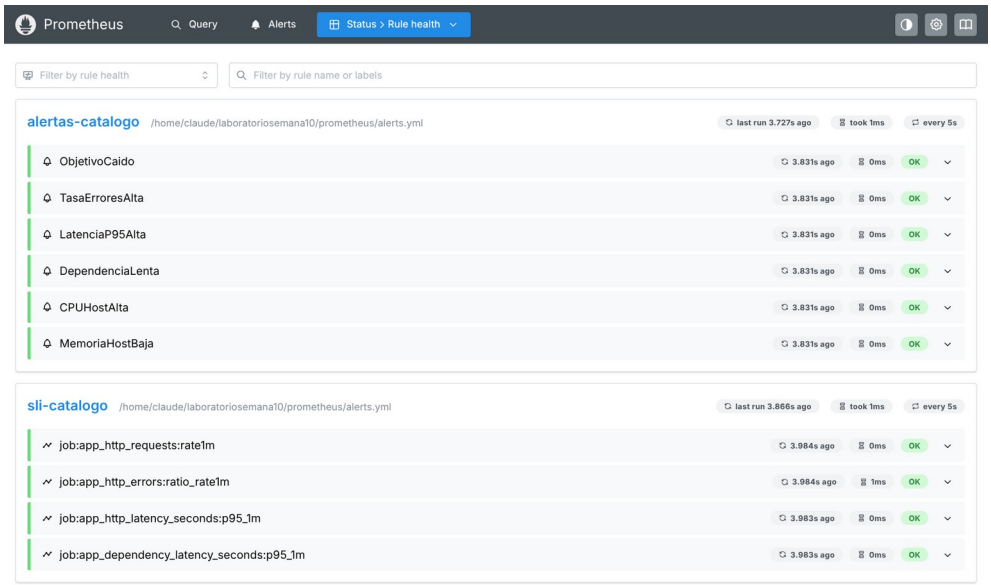


Figura 17. *Reglas cargadas en Prometheus (estado al finalizar)*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: las 4 reglas de registro y las 6 de alerta están cargadas y en estado OK, lo que confirma que la configuración se evaluó en el entorno.

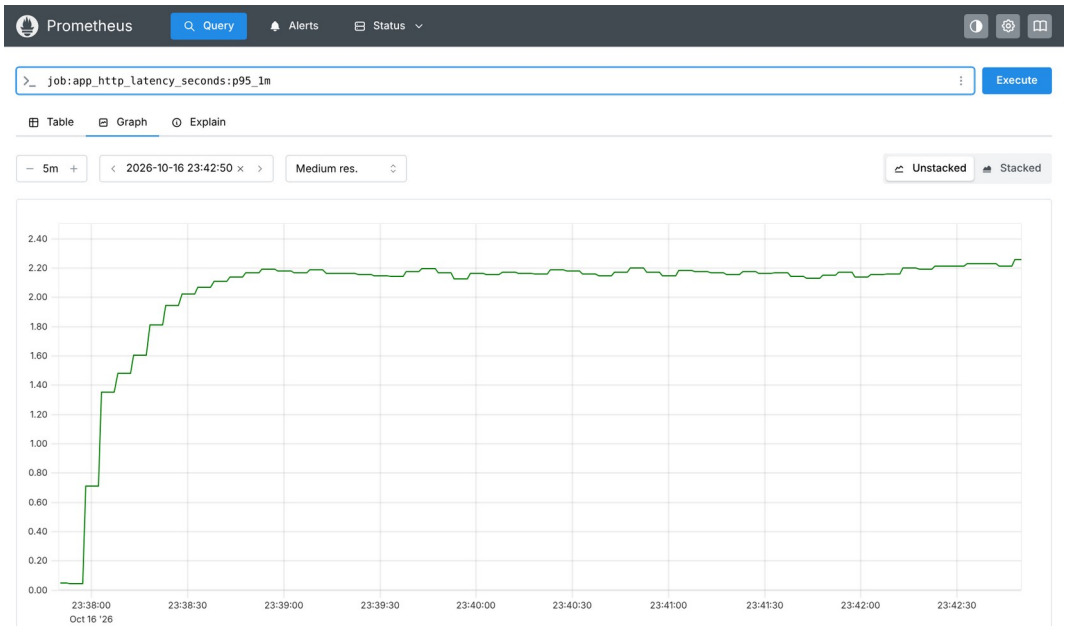


Figura 18. *Latencia p95 restringida a la ventana del incidente*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: durante los 5 minutos del fallo el p95 de 1 min se mantiene entre 2,1 y 2,3 s tras el primer minuto.

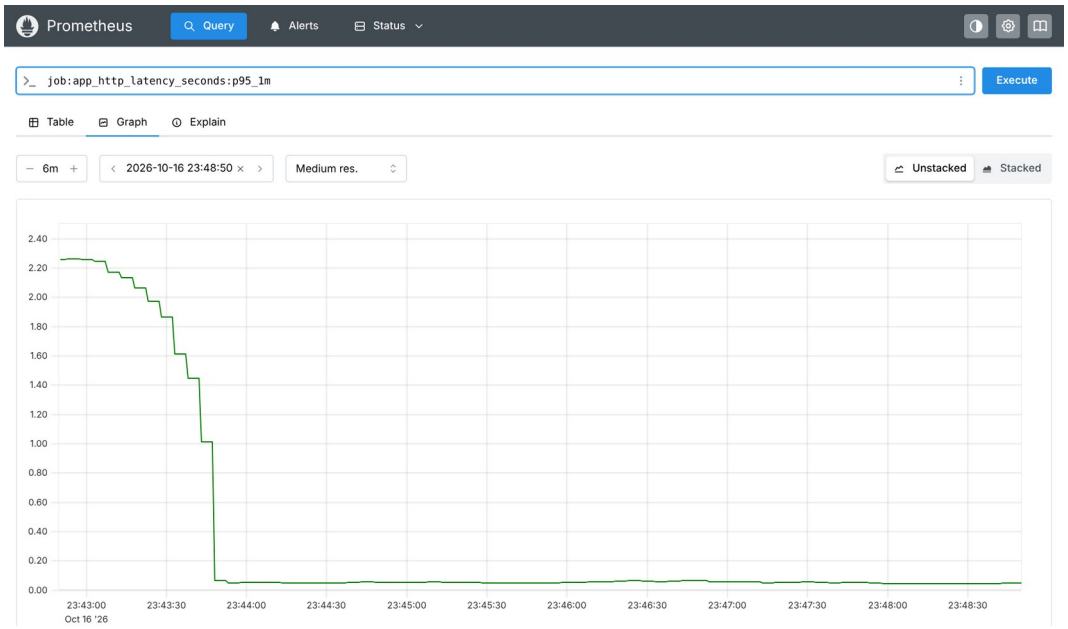


Figura 19. *Latencia p95 restringida a la ventana de recuperación*

Fuente: captura auténtica de la interfaz web de Prometheus 3.5.0 del laboratorio (eje temporal en UTC; hora de Lima = UTC − 5 h).

Interpretación: el primer minuto muestra la caída desde el valor del incidente y luego el retorno estable al rango de la línea base.